SYSTEM ANALYSIS DOCUMENT (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

This section introduces the strategic solution for SettleOps AI, an agentic platform designed to automate the complex reasoning required for insurance claim decision-making. By moving away from manual, document-heavy reviews toward a structured, multi-agent AI pipeline, SettleOps AI addresses the settlement bottlenecks identified in the motor insurance industry.

1.1. Purpose:

The system analysis document highlights the technical scope and design-related decisions behind the SettleOps AI development. It serves as a blueprint for the 9-13 specialized agents, the real-time orchestration layer, and the high-fidelity operator cockpit.

The key elements of the system that have been covered are as follows:

- **Architecture:** A stateful, agentic monolith built using FastAPI and LangGraph, implementing the PEVR (Plan, Execute, Verify, Replan) pipeline for autonomous re-analysis.
- **Data Flows:** Traces the movement of claim evidence (Police Reports, Policy PDFs, Crash Photos) from initial upload through the parallel AI reasoning layers to the final Payout and Auditor nodes.
- **Model Process:** Outlines the end-to-end workflow: document ingestion, parallel fact-checking via the PEVR loop, liability reconciliation, and human-in-the-loop audit.
- **Role of Reference:** This document is a reference for developers, AI engineers, and QA testers to ensure the system maintains high reasoning accuracy and low latency during the UMHackathon 2026.

1. 2. Background:

© UMHackathon 2026

The Malaysian motor insurance industry processes approximately 900,000 claims annually, with the "Claims Officer Review" stage acting as a severe manual bottleneck. Currently, officers spend 30-60 minutes per case mentally reconciling contradictory documents and photos. SettleOps AI was born from the need to automate this reasoning while maintaining a strict human-supervised audit trail.

- **Previous Version:** Traditional insurance systems operate as simple "document repositories" where files are stored but not understood. Reviewing each service (Policy, Police, Workshop) requires manual context-switching across multiple tabs and systems.
- **Major Architectural Changes:** SettleOps AI introduces a Stateful Intelligence Layer that treats every claim as a living "Blackboard" where independent agents write their findings in real-time.
- **New Capabilities:** Includes a Vision AI Agent for crash photo forensics and a Surgical Rerun Loop that allows the system to autonomously challenge its own logic before a human ever sees the report.

1.2. Target Stakeholder

Stakeholders	Roles	Expectations
Claims Officer	Primary operator who reviews AI-drafted decisions and provides feedback.	A high-density cockpit that surfaces contradictions and automates the 90% of routine reasoning.
Claims Team Leads / Supervisors	Monitor escalated cases, review decision quality, and use audit trails for governance.	Robust audit trails and performance metrics for claim consistency.

© UMHackathon 2026

Head of Claims Operations	Business buyer focusing on operational throughput and cost reduction.	Faster processing speeds, better consistency, and higher claim throughput.
Fleet Operations Managers / Drivers	Secondary stakeholders who benefit from fast resolution of fleet-related accidents.	Reduced vehicle downtime and clear, rapid claim settlements.

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web Application (Next.js Frontend + FastAPI Backend)
Architecture	LangGraph-Orchestrated Agentic Monolith

SettleOps AI is structured as a high-performance Agentic Platform. The system consists of a Next.js 14 Frontend providing a real-time Decision Cockpit and a FastAPI Backend that manages a stateful workflow graph. The core intelligence is handled by Google Gemini 2.5 Flash, integrated as a service layer to perform multi-agent reasoning across text and vision inputs. All case data is managed in a high-speed CaseStore and streamed to the UI via Server-Sent Events (SSE).

The architecture integrates Google Gemini 2.5 Flash as the primary reasoning layer. Instead of a generic "AI Module," the system treats the LLM as a distributed service that handles 13 specialized internal agents coordinated via the PEVR pipeline.

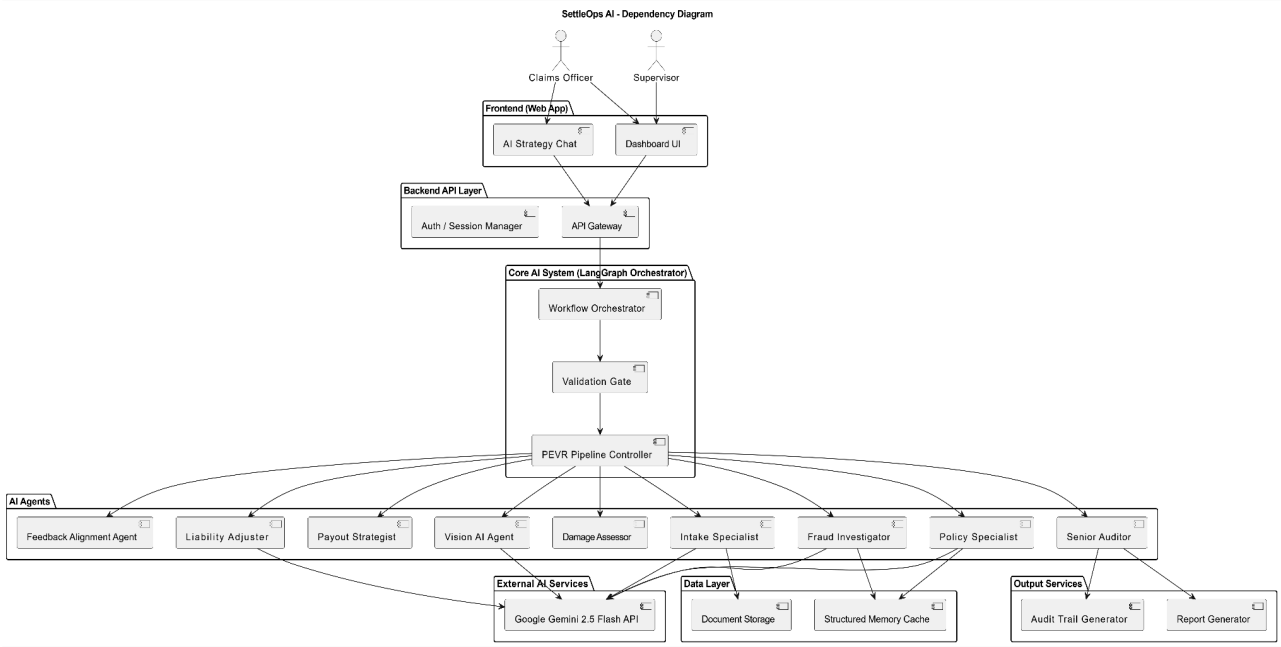
© UMHackathon 2026

2.1.2 LLM as Service Layer (Internal Agents)

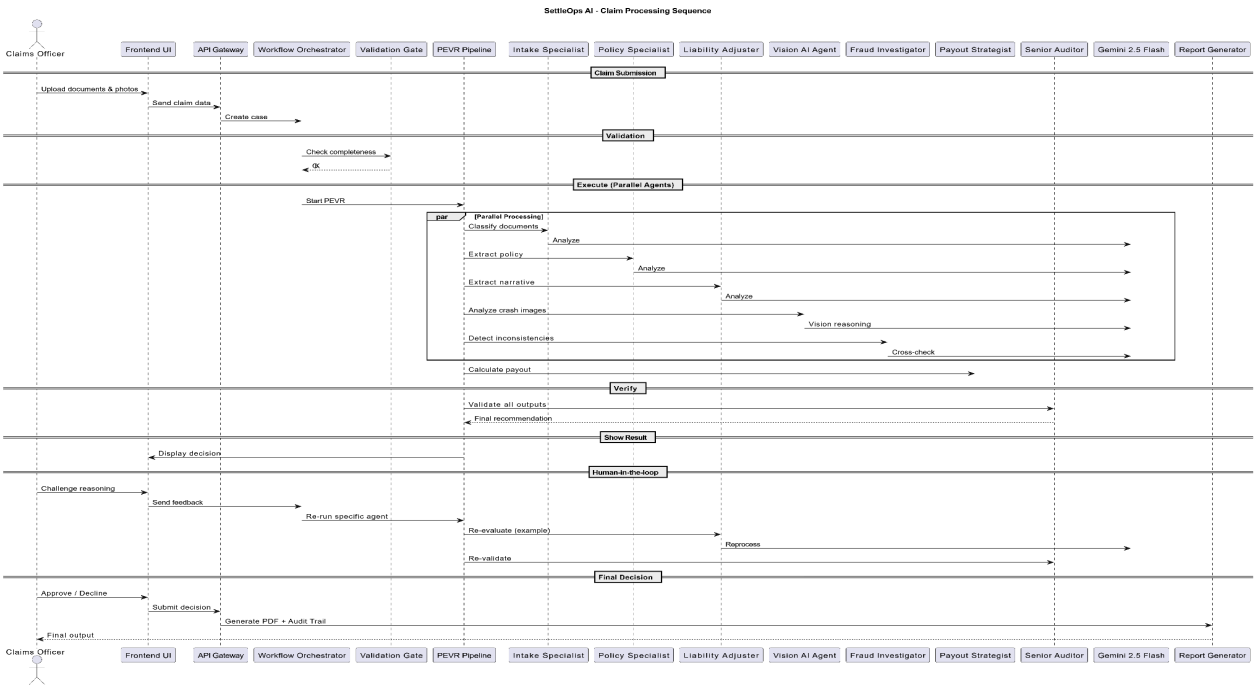
- **Intake Specialist:** Categorizes documents
- **Policy Specialist:** Extracts coverage type, maximum payouts, and excess MYR.
- **Liability Adjuster:** Extracts incident time, location, and narrative from reports.
- **Vision AI Agent:** "Visual Forensic Analyst" for Point of Impact (POI) analysis.
- **Damage Assessor:** Audits workshop repair quotes for cost and labor necessity.
- **Fraud Investigator:** Performs cross-document analysis for suspicious patterns.
- **Payout Strategist:** Deterministic agent for final MYR reconciliation.
- **Senior Auditor:** High-level supervisor for cross-consistency and rerun logic.
- **Feedback Alignment Agent:** Translates "AI Strategy Chat" into agent instructions.
- **Validation Gate:** Ensures "Intake Health" before analysis begins.
- **Report Drafter Agent:** Finalizes the decision artifact PDF with reasoning.
- **Workflow Orchestrator:** LangGraph controller managing the PEVR loops.
- **SSE Streaming Agent:** Manages real-time "Blackboard" UI updates.

2.1.3 Dependency Diagram

© UMHackathon 2026



2.2.2. Sequence Diagram



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2. Technological Stack

Layer	Technology	Rationale
Frontend Framework	Next.js 16.2.4 (App Router)	High-performance React framework for server-side rendering.
Frontend Logic	React 19.2.4	Utilizes React 19 concurrent features for smooth UI interaction.
Frontend Styling	Tailwind CSS 4	Rapid development with utility-first CSS and modern features.
Frontend State	Zustand 5	Lightweight state management for real-time dashboard state.
UI Components	React Flow, Radix UI, Lucide	For agentic graph visualization and accessible UI primitives.
Backend Framework	FastAPI, Uvicorn	High-performance async-first Python API.
Agent Orchestration	LangGraph, LangChain	Managing stateful, cyclic agent workflows with Pydantic validation.
Document Extraction	MarkItDown, PyMuPDF	Microsoft MarkItDown for structured PDF/DOCX to Markdown.

© UMHackathon 2026

Visual Forensics	Gemini Vision, Pytesseract	Gemini multimodal analysis for crash photos and OCR fallback.
Decision Artifacts	ReportLab	Programmatic generation of Claim Decision PDFs.
State Storage	In-Memory (Async-Locked)	Process-local dictionary with asyncio locking for low-latency.
Deployment	Docker	Containerized monolith for portability across environments.

3. Functional Requirements & Scope

This section defines the core MVP boundaries of SettleOps AI, focusing on high-impact features that demonstrate the feasibility of an AI agent-based automation workflow (PEVR pipeline) within the hackathon timeframe.

3.1. Minimum Viable Product:

3.1.1. Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026.

#	Features	Description
1	Claim Submission & Case Initiation	Users upload claim documents (police report, policy, adjuster report, photos) and the system validates inputs before creating a new case and triggering the AI workflow.

© UMHackathon 2026

2	Multi-Agent AI Analysis (PEVR Pipeline)	The system runs multiple specialized AI agents in parallel (liability, fraud, coverage, etc.) to analyze claim evidence and produce structured outputs within a single workflow.
3	Three-Pane Dashboard (AI Workflow Visualization)	Displays inputs, live AI reasoning stages, and structured decision outputs (blackboard) to provide transparency and real-time monitoring of the process.
4	Human-in-the-Loop Challenge & Re-analysis	Claims officers can challenge specific parts of the AI reasoning, and the system selectively reruns only the affected components using the PEVR pipeline.
5	Decision Output & Audit Trail Generation	Generates a structured Claim Decision PDF and audit trail documenting reasoning, decisions, and user interactions for compliance and review.

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system.

Quality	Requirements	Implementation
Scalability	System must handle multiple claim cases concurrently with parallel agent execution.	Use orchestrated multi-agent workflow (LangGraph-style controller) with parallel processing of agents and

© UMHackathon 2026

		modular architecture for scaling.
Reliability	AI-generated decisions must be consistent and avoid incorrect outputs or hallucinations.	Auditor Agent validation layer + human-in-the-loop control ensures all outputs are verified before finalization.
Maintainability	System components (agents, orchestrator, UI) should be modular and easy to update.	Each agent is designed as an independent module with clear responsibilities; orchestrator manages communication between them.
Performance (Latency)	Claim decision should be generated within <90 seconds under normal conditions.	Use Gemini 2.5 Flash for fast inference, parallel agent execution, and selective rerun instead of full pipeline execution.
Cost Efficiency	Token usage per claim must remain manageable for scalability.	Implement chunking, summarization, and caching of intermediate outputs; rerun only affected agents in PEVR pipeline.
Transparency & Explainability	Users must understand how decisions are made.	Three-pane dashboard + structured blackboard + AI Strategy Chat for explanation and traceability.
Security & Privacy (Basic)	Sensitive claim data should not be unnecessarily exposed.	Limit display of sensitive fields and avoid storing unnecessary personal identifiers in outputs/logs (demo-level implementation).

© UMHackathon 2026

3.3. Out of Scope / Future Enhancements

The following features represent the long-term vision of SettleOps AI but are not included in the hackathon MVP:

- **Full Integration with Insurance Ecosystems (e.g., Merimen eClaims):** Direct API integration with insurer systems to eliminate manual document uploads and enable real-time claim syncing.
- **Automated Claim Settlement (Straight-Through Processing):** Fully automated approval and payout execution for low-risk claims without human intervention.
- **Advanced Computer Vision for Damage Estimation:** AI models that can estimate repair costs directly from images and cross-check workshop quotations.
- **Cross-Insurer Fraud Detection Network:** Shared intelligence system to detect fraud patterns across multiple insurers and cases.
- **Multi-User Collaboration & Real-Time Case Sharing:** Allow multiple stakeholders (officers, supervisors, adjusters) to work on the same case simultaneously.
- **Production-Grade Security & Compliance Features:** Full authentication, role-based access control, and regulatory compliance systems.
- **Mobile Application Version:** Dedicated mobile interface for claims officers and field-related users.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. **Grayscale Rollout & A/B Testing**: The system will be deployed incrementally, starting with a small percentage of claims officers (e.g., 5%) to monitor system behavior, accuracy, and performance. Once stability is confirmed, rollout will gradually increase to full adoption. A/B testing can be conducted to compare different

© UMHackathon 2026

AI prompting strategies or agent configurations to determine which produces better accuracy and efficiency.

4.1.2. **Emergency Rollback (ER) & Golden Release**: In case of system anomalies (e.g., incorrect decision outputs or latency spikes), an Emergency Rollback mechanism will revert the system to a previously stable "Golden Version." Traffic will be redirected to this stable version while issues are investigated, ensuring minimal disruption to claim processing operations.

4.1.3. Priority Matrix:

Priority	Condition (Trigger)	Action
P1 – Critical	Incorrect or inconsistent claim decisions detected above threshold (e.g., >1% critical error rate)	Trigger immediate rollback to Golden Release and disable affected AI components
P2 – High	System latency exceeds target (e.g., >90 seconds consistently)	Optimize pipeline, reduce agent load, or limit re-analysis cycles
P3 – Medium	Individual agent failure or partial workflow breakdown	Rerun affected agent via PEVR pipeline and log issue for monitoring
P4 – Low	Minor UI or non-critical display issues	Log for future fixes, no immediate system interruption

© UMHackathon 2026

4.2. **Monitoring**:

The system will monitor:

- **Latency metrics:** Time taken per claim (target <90s)
- **Agent performance:** Success/failure rate of each agent
- **Error rates:** Inconsistencies detected by Auditor Agent
- **User actions:** Frequency of challenges and re-analysis requests

Alerts are triggered when thresholds are exceeded (e.g., high failure rates or abnormal latency), enabling rapid response and system stabilization.

4.3. Assumptions:

- Claims officers have stable internet connectivity when using the system.
- Input documents (police reports, policies, photos) are available and reasonably complete.
- Users will actively review and validate AI outputs rather than relying blindly on automation.
- The system operates in a controlled environment (demo or early deployment) without full production constraints.

4.4. **External Dependencies:**

Tools	Purpose	Risks
Google Gemini 2.5 Flash API	Core LLM for multi-agent reasoning, document understanding,	High – API downtime or rate limits may disrupt core functionality; mitigation includes fallback models or

© UMHackathon 2026

	and vision analysis	graceful degradation

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.2. Project Timeline:

Day	Activity	Key Deliverables / Focus
Day 1	Project Planning & System Design	Defined problem scope, target users, and core idea (AI agent automation workflow). Designed system architecture (multi-agent roles, PEVR pipeline, user flow). Finalized tech stack (Gemini 2.5 Flash, frontend/backend structure).
Day 2	Backend Setup & Core Pipeline Design	Set up backend structure and implemented the Workflow Orchestrator + PEVR pipeline logic. Defined agent interfaces and data flow. Started integrating Gemini API for basic prompt testing.
Day 3	Multi-Agent Implementation	Developed core reasoning agents (Intake, Policy, Liability, Fraud, etc.) and connected them to the pipeline. Implemented parallel execution logic and structured outputs for each agent.

© UMHackathon 2026

Day 4	Frontend Dashboard Development	Built the three-pane dashboard UI (Inputs, Workflow, Blackboard). Integrated backend APIs and enabled real-time updates for agent status and outputs.
Day 5	Human-in-the-Loop & Refinement Features	Implemented AI Strategy Chat, challenge mechanism, and Auditor re-analysis flow. Ensured targeted rerun (not full pipeline) works correctly. Added output generation (PDF + audit trail).
Day 6	Testing, Optimization & Demo Preparation	Performed end-to-end testing, optimized latency for sub-90s performance, fixed bugs, and prepared demo flow (including fallback mode). Finalized PRD and diagrams.

5.2. Team Members Role:

Name	Role	Responsibilities
Mah Qing Fung	Frontend Developer	Designed and implemented the dashboard UI (three-pane layout), integrated API responses, handled user interactions (challenge, approval, chat), and ensured smooth user experience.
Tan Shan Chien	Backend Developer	Developed backend APIs, implemented multi-agent pipeline logic, handled data processing, and integrated Gemini API for AI reasoning.

© UMHackathon 2026

Ivin Lee Tiew Kun	Product Manager	Defined product vision, user flow, and PRD documentation, ensured alignment with hackathon requirements, and coordinated team tasks and priorities.
Lee Keng Xiang	Full Stack Developer	Bridged frontend and backend integration, supported agent orchestration implementation, optimized system performance, and assisted in debugging and deployment.

© UMHackathon 2026